Pontifícia Universidade Católica de Minas Gerais
Curso de Engenharia de Software
Disciplina: Projeto de Software
Profa. Milena Menezes Adão

Laboratório 3 – Sprint 1

Sistema de Moeda Estudantil

Lucas Lima Magalhães Lafetá Chaves

Outubro de 2026

github.com/lucaslmchaves/sistema-moeda-estudantil

Este documento tem a modelagem do Sistema de Moeda Estudantil pedida na Sprint 1: o diagrama de casos de uso, as histórias de usuário, o diagrama de classes e o diagrama de componentes. As imagens dos diagramas também estão no repositório, na pasta `docs/diagramas`.

1. Diagrama de Casos de Uso

O sistema tem quatro atores. O aluno, o professor e a empresa parceira usam o sistema depois de fazer login. O serviço de e-mail é um sistema externo, por isso ficou do outro lado do diagrama.

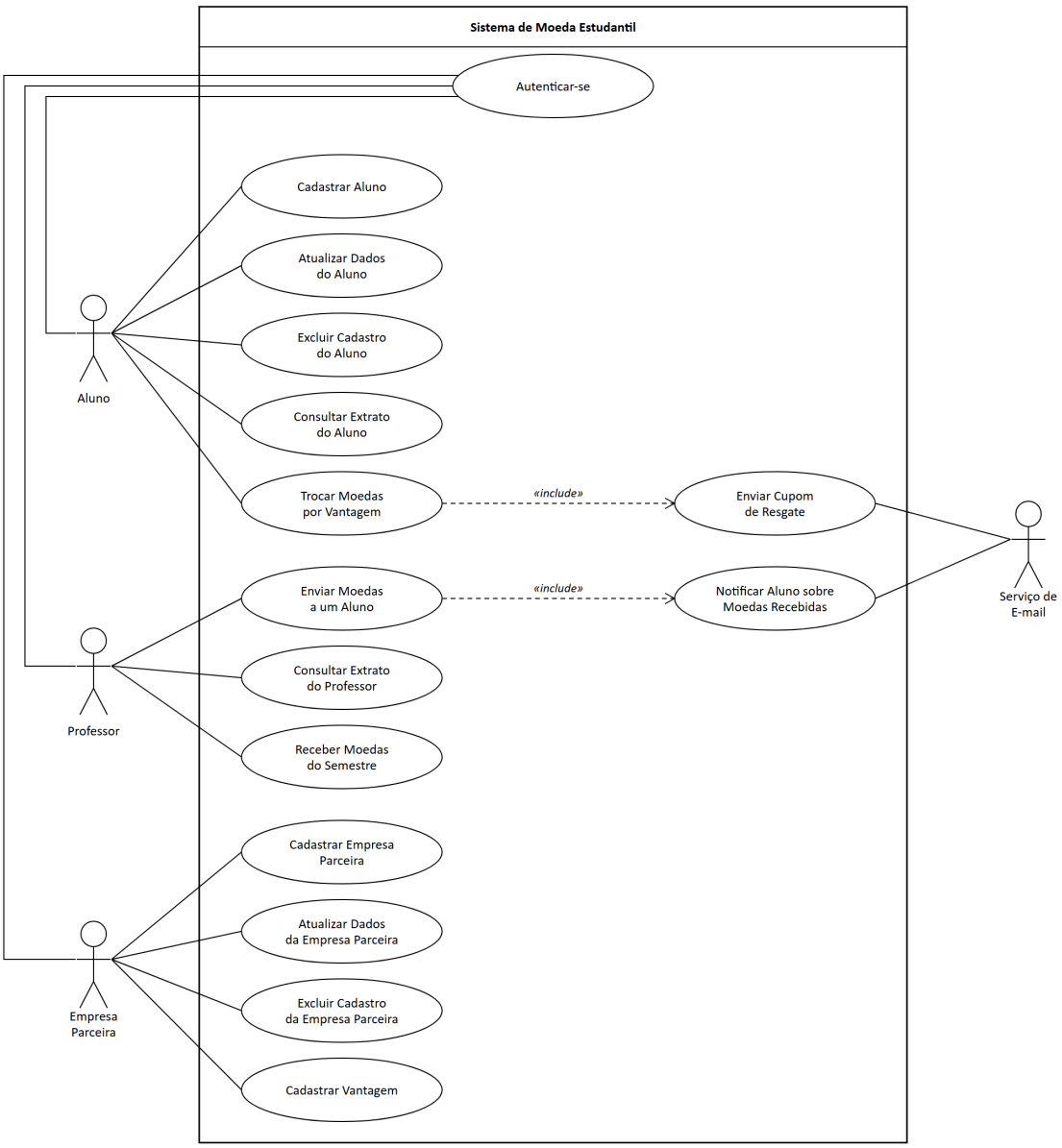


Diagrama de casos de uso

Cada caso de uso tem uma história com o mesmo nome, na seção 2. Usei «include» em dois lugares: quando o professor envia moedas o aluno sempre é avisado por e-mail, e quando o aluno

troca moedas o cupom sempre é enviado.

O caso *Receber Moedas do Semestre* acontece sozinho, porque o sistema credita as 1.000 moedas sem ninguém pedir. Mesmo assim, deixei esse caso ligado ao professor, que é quem recebe.

Instituições e professores já vêm pré-cadastrados (está no enunciado), então não fiz caso de uso para cadastrá-los. Já *atualizar* e *excluir* o cadastro de aluno e de empresa entraram porque os CRUDs são das Sprints 2 e 3.

2. Histórias de Usuário

As histórias estão separadas por ator. Os itens abaixo de cada uma são as regras do enunciado.

Todos os perfis

Autenticar-se

***Como** aluno, professor ou empresa parceira, **quero** entrar no sistema com meu login e senha, **para** acessar as funcionalidades do meu perfil.*

- Alunos, professores e empresas parceiras têm login e senha cadastrados.
- As funcionalidades de cada perfil só são acessíveis após a autenticação.
- O cadastro inicial de aluno e de empresa parceira é feito antes de existir um login.

Aluno

Cadastrar aluno

***Como** aluno, **quero** me cadastrar no sistema de mérito, **para** receber moedas dos meus professores e trocá-las por vantagens.*

- O aluno informa nome, e-mail, CPF, RG, endereço, curso, login e senha.
- A instituição de ensino é escolhida entre as instituições já pré-cadastradas.

Atualizar dados do aluno

***Como** aluno, **quero** atualizar meus dados cadastrais, **para** mantê-los corretos.*

- Só o próprio aluno autenticado altera os seus dados.

Excluir cadastro do aluno

***Como** aluno, **quero** excluir meu cadastro, **para** deixar de participar do sistema de mérito.*

- Só o próprio aluno autenticado exclui o seu cadastro.

Consultar extrato do aluno

***Como** aluno, **quero** consultar o extrato da minha conta, **para** acompanhar meu saldo e minhas movimentações.*

- O extrato mostra o total de moedas que o aluno ainda possui.
- O extrato lista os recebimentos de moedas e as trocas por vantagens.

Trocar moedas por vantagem

***Como** aluno, **quero** trocar minhas moedas por uma vantagem cadastrada, **para** aproveitar o reconhecimento que recebi.*

- O aluno seleciona uma das vantagens cadastradas no sistema.
- O saldo do aluno precisa cobrir o custo da vantagem, e o valor é descontado do saldo.
- O sistema gera um código para a troca.
- Um e-mail de cupom, com o código, é enviado ao aluno para a troca presencial.
- Um e-mail, com o mesmo código, é enviado à empresa parceira para conferir a troca.

Professor

Enviar moedas a um aluno

***Como** professor, **quero** enviar moedas a um aluno, **para** reconhecer seu bom comportamento ou sua participação em aula.*

- O professor precisa ter saldo suficiente para o montante enviado.
- O professor indica o aluno que recebe as moedas.
- O motivo do reconhecimento é uma mensagem aberta e obrigatória.
- O aluno é notificado por e-mail ao receber as moedas.

Consultar extrato do professor

***Como** professor, **quero** consultar o extrato da minha conta, **para** acompanhar meu saldo e as moedas que já enviei.*

- O extrato mostra o total de moedas que o professor ainda possui.
- O extrato lista os envios de moedas.

Receber moedas do semestre

***Como** professor, **quero** receber moedas a cada semestre, **para** ter o que distribuir aos meus alunos.*

- A cada semestre o sistema credita 1.000 moedas ao professor.
- O total é acumulável: as moedas não distribuídas permanecem no saldo e as 1.000 novas são somadas a ele.

Empresa Parceira

Cadastrar empresa parceira

***Como** empresa parceira, **quero** me cadastrar no sistema, **para** oferecer vantagens aos alunos.*

- A empresa informa nome, e-mail, CNPJ, login e senha.

Atualizar dados da empresa parceira

***Como** empresa parceira, **quero** atualizar meus dados cadastrais, **para** mantê-los corretos.*

- Só a própria empresa autenticada altera os seus dados.

Excluir cadastro da empresa parceira

***Como** empresa parceira, **quero** excluir meu cadastro, **para** encerrar a parceria com o sistema.*

- Só a própria empresa autenticada exclui o seu cadastro.

Cadastrar vantagem

***Como** empresa parceira, **quero** cadastrar as vantagens que ofereço, **para** que os alunos possam trocá-las por moedas.*

- A vantagem tem descrição, foto do produto e custo em moedas.
- A vantagem pertence à empresa que a cadastrou.

Serviço de e-mail (sistema externo)

Notificar aluno sobre moedas recebidas

***Como** serviço de e-mail, **quero** ser acionado quando um professor envia moedas, **para** avisar o aluno por e-mail.*

- O e-mail é enviado ao aluno sempre que ele recebe moedas.

Enviar cupom de resgate

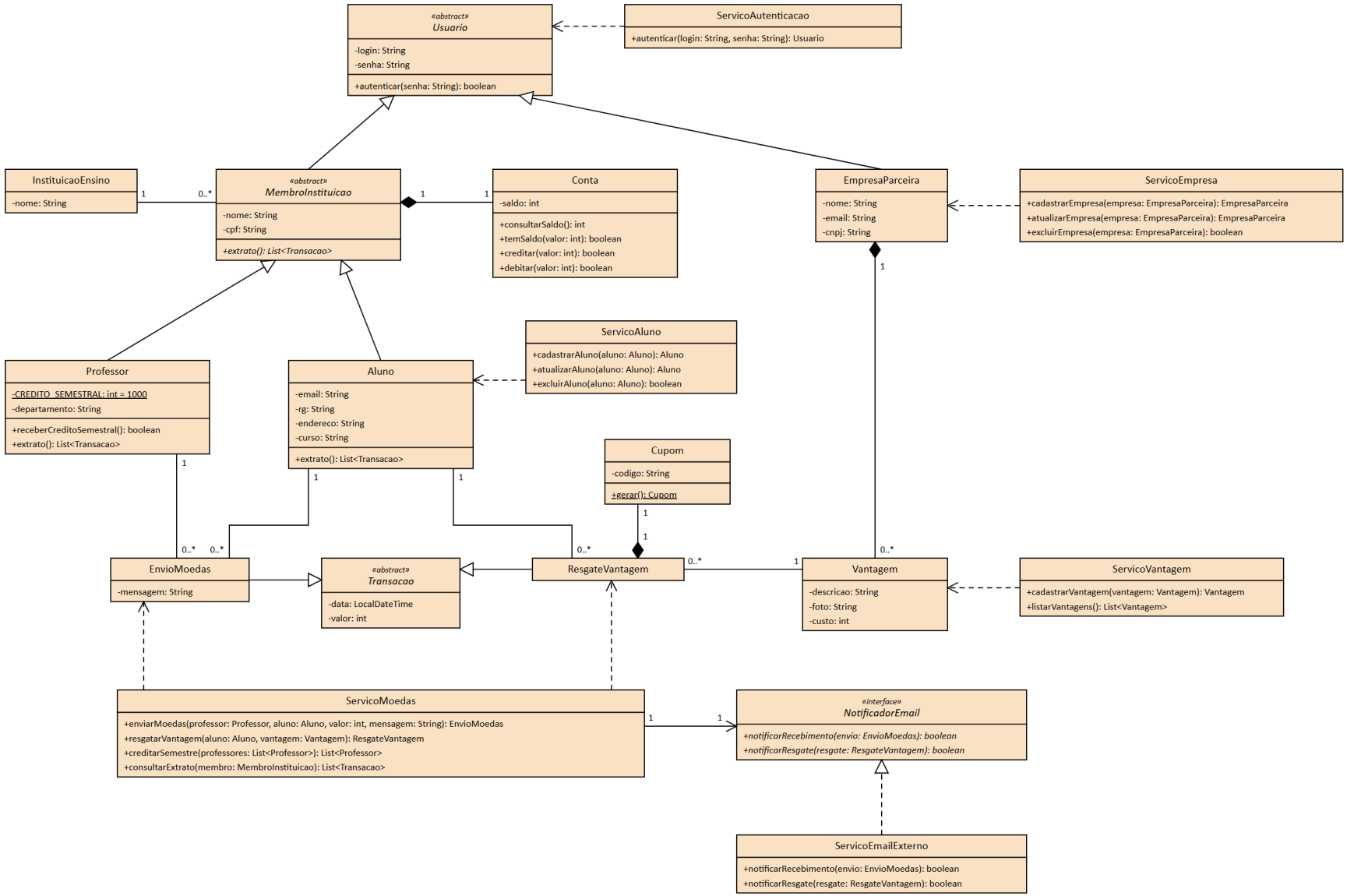
***Como** serviço de e-mail, **quero** ser acionado quando um aluno resgata uma vantagem, **para** enviar o cupom ao aluno e a conferência à empresa parceira.*

- O aluno e a empresa parceira recebem, cada um, um e-mail.
- Os dois e-mails incluem o mesmo código gerado pelo sistema.

Fora do escopo

- Instituições de ensino e professores já vêm pré-cadastrados no sistema, por isso não há história de cadastro para eles.

3. Diagrama de Classes



Como o diagrama é grande, aqui estão as principais escolhas.

Usuários. `Usuario` guarda login e senha. `Aluno` e `Professor` herdam de `MembroInstituicao`, que tem nome, CPF, a instituição e a conta. `EmpresaParceira` herda direto de `Usuario`, já que a empresa não pertence a uma instituição.

Saldo. Em vez de repetir o saldo em `Aluno` e `Professor`, criei a classe `Conta`, com `consultarSaldo`, `temSaldo`, `creditar` e `debitar`. A regra de saldo suficiente fica só nela.

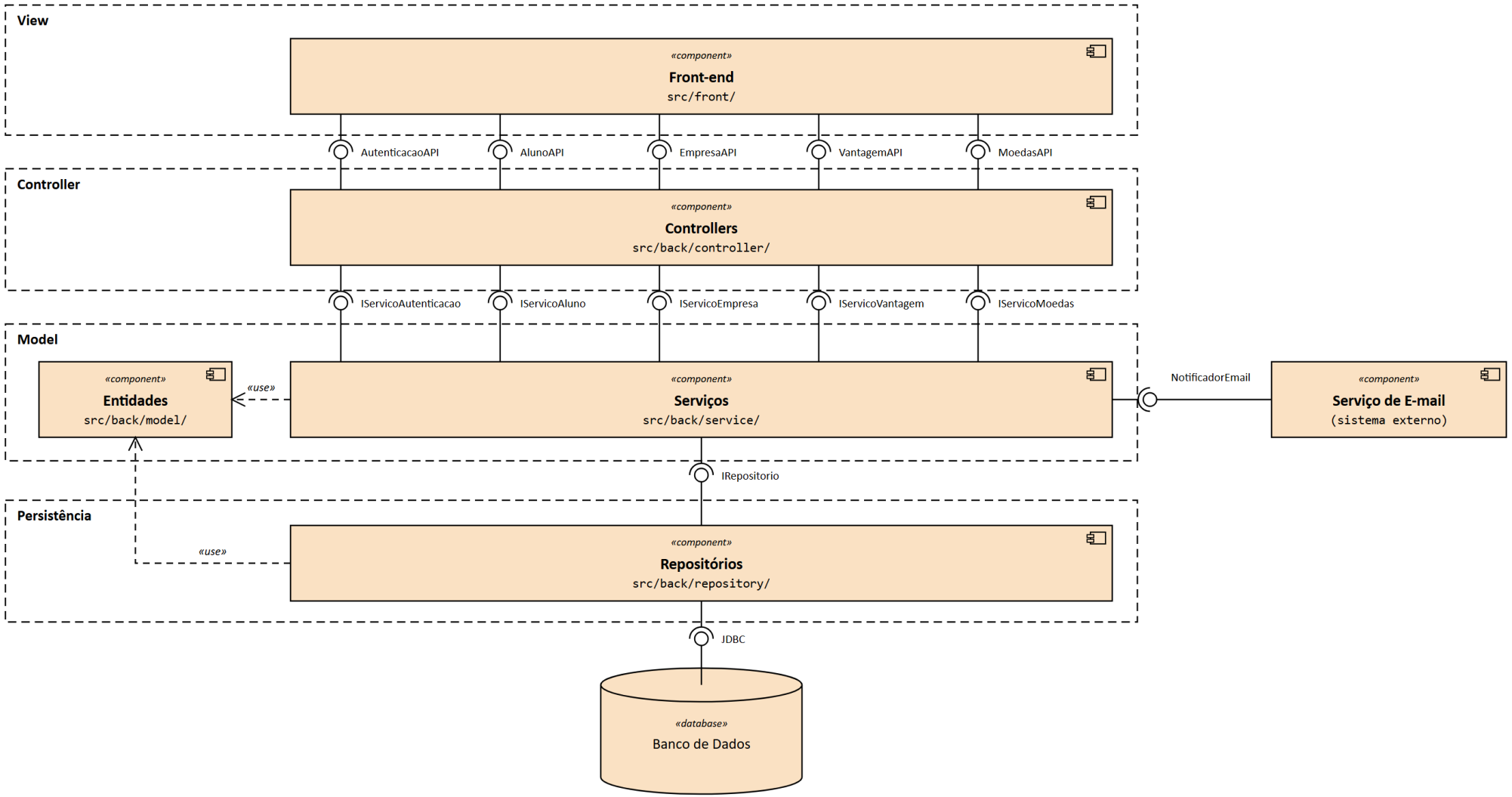
Transações. `Transacao` é abstrata, com data e valor. `EnvioMoedas` é o envio do professor para o aluno, com a mensagem obrigatória, e `ResgateVantagem` é a troca do aluno por uma vantagem. O resgate tem um `Cupom`, com o código gerado pelo sistema que vai nos dois e-mails. Assim, as duas relações muitos-para-muitos (professor e aluno, aluno e vantagem) viraram classes.

Regras do enunciado. As 1.000 moedas por semestre são a constante `CREDITO_SEMESTRAL` do `Professor`. O enunciado não fala de nenhum outro limite numérico, por isso as cardinalidades são só 1 e 0..*. A mensagem obrigatória é o atributo `mensagem` do `EnvioMoedas`, e o saldo suficiente é verificado com `Conta.temSaldo`.

Serviços. Separei por assunto: `ServicoAutenticacao`, `ServicoAluno`, `ServicoEmpresa`, `ServicoVantagem` e `ServicoMoedas` (enviar moedas, resgatar vantagem, creditar o semestre e consultar o extrato). O `ServicoMoedas` usa a interface `NotificadorEmail` para avisar o aluno e mandar os cupons, e quem realiza essa interface é o `ServicoEmailExterno`. Assim a regra de negócio não fica presa a um provedor de e-mail.

O diagrama mostra só os atributos e as operações públicas de negócio. Não coloquei construtores, getters, métodos privados nem classes de persistência, e nenhuma operação retorna `void`.

4. Diagrama de Componentes



A arquitetura é MVC, como o enunciado pede. A View é o front-end (`src/front/`). O Controller recebe as requisições e chama os serviços (`src/back/controller/`). O Model reúne os serviços (`src/back/service/`) e as entidades do diagrama de classes (`src/back/model/`). A persistência ficou em uma camada separada, com os repositórios (`src/back/repository/`) e o banco de dados. Como a escolha entre ORM e DAO só acontece na Sprint 2, ela aparece aqui de forma geral.

Os componentes conversam por interfaces: o front-end usa as APIs dos controllers, os controllers usam uma interface de cada serviço e os serviços usam o `IRepositorio`. O front-end nunca fala direto com o banco. O serviço de e-mail é externo e se liga aos serviços pela interface `NotificadorEmail`, a mesma do diagrama de classes. Serviços e repositórios também usam as entidades (seta tracejada «use»).

Interface	**Fornecida por**	**Requerida por**
`AutenticacaoAPI`, `AlunoAPI`, `EmpresaAPI`, `VantagemAPI`, `MoedasAPI`	Controllers	Front-end
`IServicoAutenticacao`, `IServicoAluno`, `IServicoEmpresa`, `IServicoVantagem`, `IServicoMoedas`	Serviços	Controllers
`IRepositorio`	Repositórios	Serviços
`JDBC`	Banco de Dados	Repositórios
`NotificadorEmail`	Serviço de E-mail (externo)	Serviços